

MixBytes()

[SECURITY AUDIT REPORT]

Lido TokenRateNotifier

PREPARED FOR LIDO

REPORT DATE

August 04, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	10
2.1 Critical	10
2.2 High	10
2.3 Medium	10
2.4 Low	11
L-1 ObserverAdded / ObserverRemoved omit the observer kind	11
3. About MixBytes	12

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

`TokenRateNotifier` is the L1 fan-out hub that is registered in `LidoLocator` as Lido's `postTokenRebaseReceiver`, so that Lido core (`Accounting`) invokes it exactly once per oracle report to broadcast the rebase event. It maintains a governance-curated registry of observers and forwards each rebase to them, isolating individual failures in a `try/catch` so one broken observer cannot block the report. This review covers an extension that introduces a second observer flavor: alongside the existing zero-argument `NoArgs` observers (L2 rate pushers that self-fetch `wstETH.stEthPerToken()`), a new `WithArgs` flavor receives the full per-rebase payload - most notably `sharesMintedAsFees` - enabling downstream consumers such as revenue/buyback engines to act on values that cannot be back-derived from a rate delta. Both flavors coexist in a single registry (capped at 32 entries), keyed by kind and validated at registration through ERC165.

The interim security review was conducted over 2 working days via manual code review and a proprietary AI-assisted analysis tool.

During the audit, our focus covered both the core notifier logic and its interactions with the surrounding integrations - the upstream trusted provider that drives `handlePostTokenRebase` and the downstream observer contracts invoked during fan-out. Beyond that, we worked through the standard internal checklist for this class of contract: access control, reentrancy, arithmetic and rounding behavior, storage packing correctness, and the revert-propagation semantics that determine whether a failing observer can halt the oracle report. The project-specific vectors examined during this engagement are enumerated below.

The `WithArgs` consumer `StakingRevenueSource`, as well as deploy/upgrade scripts and tests and mocks, were out of scope. Where the notifier's safety depends on the behavior of downstream observers, our assessment was limited to the integration assumptions visible from the in-scope code and the relevant interfaces, not a line-by-line audit of those components.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **The diff is small and correct.** Storage packing is sound, the two interface IDs are distinct, swap-and-pop removal and payload forwarding are correct, and there are no memory-safety or access-control defects. Reentrancy is not reachable, since an observer can call neither `addObserver` / `removeObserver` (`onlyOwner`) nor `handlePostTokenRebase` (`onlyProvider`).
- **The one architectural concern is a pre-existing, accepted trade-off, not a bug in this diff.** `Accounting` calls `handlePostTokenRebase` directly, with no `try/catch`. The notifier does `try/catch` each observer, but intentionally re-reverts on empty returndata (assumed out-of-gas), so such a failure escapes the inner catch, propagates through the unguarded outer call, and halts the oracle report. This mechanism predates the change; the diff only widens its exposure by routing a more complex `WithArgs` consumer onto that path.

We recommend an explicit callback contract for `WithArgs` observers: `pushTokenRate(...)` must never empty-revert and must be gas-bounded.

- **The ownership recovery lever is flagged for attention, not as a finding.** `removeObserver` is the only on-chain remedy against a halting observer and is `onlyOwner`; via inherited OZ `Ownable`, renouncing ownership would freeze recovery permanently. We recommend overriding `renounceOwnership` to revert.
- **Payload forwarding is correct and pushes obligations onto the consumer.** The notifier forwards all seven values verbatim without validation, which is correct since they come from the trusted provider. The `WithArgs` consumer must therefore handle `_sharesMintedAsFees == 0` without empty-reverting. As the consumer keeps its own copy of `ITokenRatePusherWithArgs`, a CI check that the two signatures stay identical is advisable.
- **`WithArgs` raises worst-case fan-out gas while the observer cap is unchanged.** A `WithArgs` notification forwards seven `uint256` values (224 bytes) and typically drives heavier observer logic than the zero-argument `NoArgs` flavor, yet `MAX_OBSERVERS_COUNT` stays at 32. Since the report path has no gas cap, the safe headroom before a full fan-out risks out-of-gas is reduced relative to the `NoArgs`-only design. We recommend re-validating the worst-case gas of a full 32-observer `WithArgs` fan-out against the oracle report's gas budget, and considering a lower cap for the heavier flavor.
- **The fan-out has no per-observer gas isolation.** Each observer is called with all remaining gas, so a single observer that consumes nearly all of it can starve those that follow and, via the empty-data re-revert, take down the whole report. This is not exploitable under the DAO-curated trust model, but it is the structural root of the halting-report fragility above. We recommend considering a bounded per-observer gas stipend, weighed against the gas-estimation rationale in `_handlePushFailure`.
- **Each observer supports exactly one flavor.** An address is registered at most once: `addObserver` rejects a duplicate address regardless of `kind` (`_observerIndex` check), and `removeObserver` matches purely by address. The design therefore assumes an observer is either `NoArgs` or `WithArgs`, never both - registering one address under both flavors is not possible, and address-keyed removal would be ambiguous if it were. This assumption is sound as implemented.

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	TokenRateNotifier
TYPE	Solidity
PLATFORM	EVM
TIMELINE	07.07.2026 - 13.07.2026

Scope of Audit

`contracts/0.8.9/TokenRateNotifier.sol``contracts/0.8.9/interfaces/ITokenRatePusherWithArgs.sol``contracts/0.8.9/interfaces/ITokenRatePusher.sol`

Versions Log

DATE	COMMIT HASH	NOTE
07.07.2026	1572a1a4d9ff0efb9ff9aa2500facc67b7310dfc	Initial Commit
13.07.2026	dc9066b6f9b5c6c3b65f4a4aac338427198f0e98	Commit for Re-audit

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
TokenRateNotifier.sol	0xbe05d12FD10919F1881125006523452F6aFf791b	Ethereum

The deployment compiles byte-for-byte from the audit-scope source at commit `dc9066b6f9b5c6c3b65f4a4aac338427198f0e98`. Bytecode verification performed against on-chain state at block 25545333. Both the creation bytecode and the runtime bytecode match. The only deviations in the runtime code are the two `TOKEN_RATE_PROVIDER` immutable placeholders, both holding the expected value.

The contract was created by transaction

`0xa031ffa7509f7c253704915e3af973ae54057c53fb5086023792f961da52babc` from the EOA `0xc0d0B847F3E0Cb87C68eD02208C01aA83b1800E2`. The appended constructor arguments decode to `initialOwner_ = 0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c` (Lido Aragon Agent) and `tokenRateProvider_ = 0x23ED611BE0E1a820978875C0122F92260804CDdf` (Lido Accounting). Both are correct: the Accounting instance reports `LIDO() = 0xae7ab96520DE3A18E5e111B5EaAb095312D7fE84` (stETH) and `LIDO_LOCATOR() = 0xC1d0b3DE6792Bf6b4b37EccdcC24e45978Cfd2Eb` (canonical `LidoLocator`), and `LidoLocator.accounting()` resolves back to the same address, so `TOKEN_RATE_PROVIDER` is the genuine, single authorized caller of `handlePostTokenRebase`. The two `OwnershipTransferred` events in the creation transaction (`0x0 -> deployer -> Agent`) are the constructor's `_transferOwnership`; the deploying EOA retains no privilege over the contract.

On-chain state is the constructor baseline only. `owner()` is the Agent, `observersLength()` is `0`, and the constants read back as audited: `MAX_OBSERVERS_COUNT = 32`, `INDEX_NOT_FOUND = type(uint256).max`. The only events ever emitted by the contract are the two from the creation transaction - no `ObserverAdded` has been issued. Access control was exercised against live state: `handlePostTokenRebase` called from an arbitrary sender reverts with `ErrorNotAuthorizedRebaseCaller()`, and succeeds as a no-op when simulated from Accounting.

Two items require re-validation after the Lido DAO vote, since the deployment is staged but not yet live:

The notifier is not yet wired into the protocol. `LidoLocator.postTokenRebaseReceiver()` still resolves to the previous notifier `0x25E35855783BEC3E49355A29e110F02Ed8b05bA9`, which is the pre-audit `NoArgs`-only implementation.

The observer registry is empty. The old notifier carries one observer, `0xd54C1c6413CAAc3477AC14B2a80D5398e3C32ffe`, which declares support for `ITokenRatePusher` and not for `ITokenRatePusherWithArgs`, i.e. it is a `NoArgs` observer that must be re-registered on the new contract with `kind = NoArgs` to preserve current behavior. Re-validation will be required to confirm that each observer is registered under the flavor matching the interface it actually implements.

Ownership is exercised through Lido DAO governance rather than a multisig: the owner `0x3e40D73EB977Dc6a537aF587D48316feE66E9C8c` is an Aragon `AppProxyUpgradeable`, so `addObserver` / `removeObserver` are reachable only via a passed DAO vote. Note that this owner is also the only on-chain remedy against a halting observer, and that the inherited OZ `renounceOwnership` remains callable by it.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

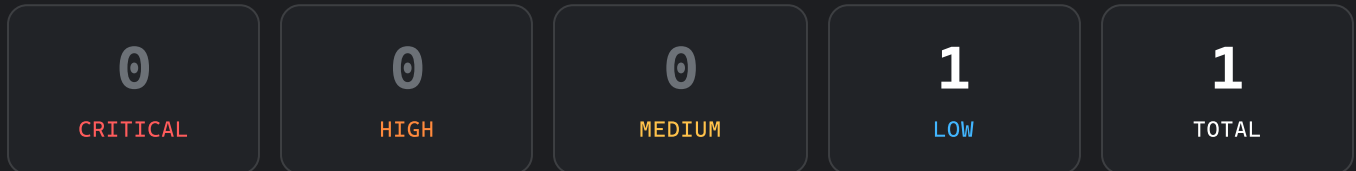
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
L-1	ObserverAdded / ObserverRemoved omit the observer kind	● LOW	● FIXED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 `ObserverAdded / ObserverRemoved` omit the observer kind

● LOW

● FIXED AS OF dc9066b6

DESCRIPTION

`ObserverAdded(address)` and `ObserverRemoved(address)` do not carry the observer flavor. Now that `kind` is a first-class dimension of the registry, off-chain indexers and monitors cannot reconstruct which observers are `WithArgs` without reading storage, which weakens monitoring of the exact observers that sit on the protocol-halting path.

RECOMMENDATION

Add `ObserverKind` (indexed) to `ObserverAdded`. Additionally, consider including the report timestamp in `PushTokenRateFailed` so a failed `WithArgs` push can be tied to a specific oracle report.

CLIENT'S COMMENTARY

`PushTokenRateFailed` fires inside the oracle-report tx, and `Lido.TokenRebased` in that same tx already carries `reportTimestamp`. Indexers can tie a failed push to its report by joining on the transaction, so we don't duplicate the value in the notifier event. `ObserverAdded/Removed` now carry the kind.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[!\[\]\(1f99bf65f43889da445ecc1fe8d9504f_img.jpg\)](#) [!\[\]\(a7701fd885e07c9a2352fce4252da8ed_img.jpg\)](#) [!\[\]\(03c45162207ca863157643ba63c566c6_img.jpg\)](#) [!\[\]\(dcf280f23c5b802c3d4fe47e911df364_img.jpg\)](#)